

# Programowanie obiektowe w Javie

## 1. Czym jest programowanie obiektowe?

Programowanie obiektowe (OOP — Object-Oriented Programming) to sposób tworzenia programów, w którym organizujemy kod za pomocą obiektów.

Obiekt może posiadać:

- dane — czyli informacje o nim,
- metody — czyli czynności, które może wykonywać.

Przykładowo możemy stworzyć obiekt **Samochod**.

Samochód może mieć dane:

```
String marka;  
String kolor;  
int predkosc;
```

oraz wykonywać czynności:

```
jedz();  
hamuj();  
przyspiesz();
```

Czyli w dużym uproszczeniu:

obiekt = dane + funkcje (metody)

---

## 2. Klasa a obiekt

To jedna z najważniejszych rzeczy do zrozumienia. Klasa jest projektem/szablonem obiektu.

Na przykład:

```
class Samochod {  
    String marka;  
    String kolor;  
    int predkosc;  
}
```

Ta klasa mówi, że każdy samochód będzie miał:

- markę,
- kolor,
- prędkość.

Ale sama klasa nie jest jeszcze konkretnym samochodem.

## Obiekt

Obiekt jest konkretnym egzemplarzem klasy. Możemy stworzyć go za pomocą słowa **new**:

```
Samochod samochod1 = new Samochod();
```

Teraz **samochod1** jest obiektem klasy **Samochod**.

Możemy stworzyć wiele obiektów tej samej klasy:

```
Samochod samochod1 = new Samochod();  
Samochod samochod2 = new Samochod();  
Samochod samochod3 = new Samochod();
```

Każdy z nich może mieć inne dane.

---

## 3. Tworzenie klasy

Przykładowa klasa:

```
class Osoba {  
    String imie;  
    int wiek;  
}
```

Możemy teraz utworzyć obiekt:

```
Osoba osoba1 = new Osoba();
```

A następnie ustawić jego dane:

```
osoba1.imie = "Jan";
```

```
osoba1.wiek = 18;
```

Możemy również stworzyć drugą osobę:

```
Osoba osoba2 = new Osoba();
```

```
osoba2.imie = "Kasia";  
osoba2.wiek = 20;
```

Mamy więc dwa różne obiekty:

```
osoba1  
  imie = Jan  
  wiek = 18
```

```
osoba2  
  imie = Kasia  
  wiek = 20
```

Oba obiekty są stworzone na podstawie tej samej klasy **Osoba**.

---

## 4. Konstruktor

Często chcemy od razu podczas tworzenia obiektu ustawić jego dane.

Do tego służy konstruktor.

Konstruktor to specjalna metoda, która jest uruchamiana podczas tworzenia obiektu.

Przykład:

```
class Osoba {  
    String imie;  
    int wiek;  
  
    Osoba(String imie, int wiek) {  
        this.imie = imie;  
        this.wiek = wiek;  
    }  
}
```

Teraz możemy stworzyć osobę:

```
Osoba osoba1 = new Osoba("Jan", 18);  
Osoba osoba2 = new Osoba("Kasia", 20);
```

Nie musimy już robić:

```
Osoba osoba1 = new Osoba();  
  
osoba1.imie = "Jan";  
osoba1.wiek = 18;
```

Konstruktor zrobi to za nas.

---

## 5. Co oznacza **this**?

W konstruktorze mamy:

```
this.imie = imie;
```

Słowo **this** oznacza aktualny obiekt. Mamy tutaj dwie zmienne o nazwie **imie**:

```
this.imie  
to pole obiektu, a:  
imie
```

to parametr konstruktora.

Czyli:

```
this.imie = imie;
```

oznacza:

ustaw pole **imie** aktualnego obiektu na wartość przekazaną do konstruktora.

Podobnie:

```
this.wiek = wiek;
```

---

## 6. Metody — funkcje w klasach

W Javie funkcje znajdujące się wewnątrz klasy nazywamy najczęściej metodami.

Przykład:

```
class Osoba {  
    String imie;  
    int wiek;  
  
    void przedstawSie() {  
        System.out.println("Mam na imie " + imie);  
    }  
}
```

Możemy wywołać metodę:

```
Osoba osoba1 = new Osoba();  
osoba1.imie = "Jan";  
  
osoba1.przedstawSie();
```

Wynik:

Mam na imie Jan

Metoda może również korzystać z danych obiektu.

---

## 7. Metody z parametrami

Metoda może przyjmować parametry.

Przykład:

```
class Kalkulator {  
  
    void dodaj(int a, int b) {  
        System.out.println(a + b);  
    }  
}
```

Tworzymy obiekt:

```
Kalkulator kalkulator = new Kalkulator();
```

Wywołujemy metodę:

```
kalkulator.dodaj(5, 3);
```

Wynik:

8

---

## 8. Metody zwracające wartość

Metoda nie musi tylko coś wyświetlać. Może również zwracać wynik za pomocą **return**.

Przykład:

```
class Kalkulator {  
  
    int dodaj(int a, int b) {  
        return a + b;  
    }  
}
```

Możemy zrobić:

```
Kalkulator kalkulator = new Kalkulator();
```

```
int wynik = kalkulator.dodaj(5, 3);
```

```
System.out.println(wynik);
```

Wynik:

8